

PROJECT REPORT

Automation of an E-Commerce Web Application Using Selenium WebDriver with Python

Capstone Project 1

Field	Detail
Name	Mayukh Ghosh
Enrollment No.	12023002022242
Stream	CST
Project	Capstone Project 1: Automate a Web Application Using Selenium WebDriver with Python

GitHub Link: https://github.com/Mayukh27/Wipro_Python_Automation

Video Demonstration Link: https://drive.google.com/drive/folders/13hHv1CmXM-gd4oN5LhwraoYLVVpvDJHe?usp=drive_link

Field	Detail
Application Under Test	https://automationexercise.com/ (public e-commerce practice website)
Automation Tool	Selenium WebDriver 4.x
Programming Language	Python 3.x
Design Approach	Modular, function-based Python script (one function per business step); no external test framework
Browser	Google Chrome (driver resolved automatically by Selenium Manager)
Date	September 2026

Capstone Project 1: Automate a Web Application Using Selenium WebDriver with Python

Business Scenario

A customer wants to purchase a product from an e-commerce site. The automation should:

1. Launch browser
2. Login to application
3. Search product
4. Add product to cart
5. Update quantity
6. Verify cart details
7. Capture screenshots
8. Read test data from Excel/JSON
9. Handle popup/alerts if available
10. Generate execution report

1. Introduction

This report documents the design, implementation and execution of an automated test that simulates a customer purchase journey on a public e-commerce practice application. The solution is written in Python with Selenium WebDriver 4 and is deliberately kept simple and readable: a single script whose functions map one-to-one to the steps of the business scenario.

The goal is to demonstrate practical browser automation skills: reliable locators, explicit synchronization, assertions that verify real application data, evidence capture through screenshots, handling of alerts and overlays, and structured execution reporting.

1.1 Business Scenario

A customer opens the shop, signs in, searches for a product, adds it to the cart, changes the quantity of a second product, and reviews the cart to confirm that names, quantities and prices are correct. The automation reproduces this journey end to end and verifies the outcome of every step.

1.2 Objectives

- Launch Chrome and open the application under test.
- Authenticate into the application (register the account on the first run, log in directly on later runs).
- Search for a product using the site search.
- Add a located product to the cart.
- Update the quantity of a product before adding it to the cart.
- Verify that the cart shows the correct products, quantities and line totals.
- Capture a screenshot after every functional step.
- Drive the run from a single test-data block instead of values scattered through the code.
- Detect and handle JavaScript alerts and in-page popups when they appear.
- Generate an HTML execution report summarizing the run.

2. Tools and Technologies

Category	Technology / Tool
Programming Language	Python 3.x
Automation Library	Selenium WebDriver 4.x (Selenium Manager resolves the ChromeDriver automatically)
Test Runner	None required: the script is run directly with Python and uses assert statements
Design Approach	Function-based modular script: locators, utilities, step functions, reporting, orchestration
Test Data	TEST_DATA dictionary at the top of the script
Synchronization	WebDriverWait with expected_conditions (no time.sleep)
Reporting	Custom self-contained HTML report generator
Dependencies	selenium (single entry in requirements.txt)
Application Under Test	automationexercise.com

2.1 Choice of Application

Automation Exercise was chosen as the simplest reliable workflow for this project. It is built for automation practice, exposes stable data-qa attributes on its forms, has no captcha, and publishes official test scenarios. The quantity workflow used in this project follows its official scenario for verifying product quantity in the cart, where the quantity is increased on the product detail page and then verified in the cart.

3. Project Architecture

Although everything lives in one file, the code is organised in clearly separated layers so that a change in the website only affects the locators, and the business flow in `main()` still reads as a plain narrative.

Layer	Responsibility	Contents
Configuration and locators	URLs, output folders, test data and all element locators kept in one place	BASE_URL, SCREENSHOT_DIR, REPORT_DIR, TEST_DATA, locator tuples
Utilities	Low-level browser, evidence and reporting helpers	create_driver, safe_click, take_screenshot, handle_alert_if_present, record, parse_rupee_amount
Step functions	One function per business step with its own assertions	step_login, step_search_product, step_add_product_to_cart, step_update_quantity_and_add_second_product, step_verify_cart_details
Reporting	Builds the HTML execution report from recorded steps	write_html_report
Orchestration	Runs the steps in order, handles failure and always cleans up	main()

4. Project File Structure

```
capstone_assignment_1/
|-- capstone_assignment_1.py      Complete automation script
|-- requirements.txt             Dependencies (selenium)
|-- screenshots/                 Created automatically at run time
`-- reports/
    |-- execution_report.html     Generated automatically at run time
```

5. Test Data Management

All inputs are kept in the TEST_DATA dictionary at the top of the script. The step functions contain no literal search terms, quantities or profile values, so the scenario can be changed by editing one block.

Data item	Purpose
search_keyword	Keyword typed into the product search box ("Top")
product_a_index	Position of the search result added with the default quantity
product_b_index	Position of the search result whose quantity is updated
quantity_to_set	Quantity entered for the second product (4)
profile	Registration and login details: name, email, password, date of birth, address, country, state, city, ZIP code and mobile number

Products are selected by their position in the search results, and their names are read from the page at run time. This keeps the test independent of the exact catalogue content. The dictionary keeps the same structure as a JSON document, so moving it to an external JSON or Excel file only requires replacing the dictionary with a file-loading call.

6. Automated Execution Flow

main() executes the following steps in sequence. Each step asserts its own result, captures a screenshot and records a PASS or FAIL entry that is later written to the report.

#	Step	Key actions and verification
1	Launch Browser	Starts Chrome with a 1600x1000 window and opens the home page. Verifies the page title contains "Automation Exercise".
2	Login (step_login)	Opens the login page and submits name and email in the signup form. If the site reports that the email already exists, logs in directly with the stored credentials; otherwise completes the account information form (password, date of birth, name, address, country, state, city, ZIP, mobile), creates the account and continues. Verifies that the header shows "Logged in as".
3	Search Product (step_search_product)	Opens the Products page, types the keyword and submits the search. Verifies the "Searched Products" heading and that at least two results were found; collects the result links.

#	Step	Key actions and verification
4	Add Product to Cart (step_add_product_to_cart)	Opens the first result, reads the product name, clicks Add to cart with the default quantity. Checks for an alert, waits for the confirmation modal and opens View Cart.
5	Update Quantity (step_update_quantity_and_added_second_product)	Opens the second result, clears the quantity field and enters the value from TEST_DATA. Asserts the field really holds the new value, adds the product to the cart and opens the cart.
6	Verify Cart Details (step_verify_cart_details)	Reads every row of the cart table: product name, unit price, quantity and line total. Asserts both products are present, quantities are 1 and 4, and that line total equals unit price multiplied by quantity.
7	Screenshot Capture	A screenshot is saved after each step; a failure screenshot is saved if any step raises an error.
8	Read Test Data	Search keyword, product positions, quantity and registration profile all come from TEST_DATA.
9	Alert / Popup Handling	handle_alert_if_present() is called after each Add to cart click.
10	Generate Report	write_html_report() builds execution_report.html in a finally block so it is produced even when a step fails.

7. Alert, Popup and Overlay Handling

7.1 JavaScript alerts

handle_alert_if_present() waits briefly for a native alert, confirm or prompt dialog. If one appears it reads the text, accepts or dismisses it and returns the text, which is then logged in the report. If none appears it returns None and the run continues, so the check never fails a test by itself. In the add-to-cart flow this site shows an in-page confirmation modal instead of a native alert, and that modal is handled explicitly through its View Cart link.

7.2 Advertisement overlays

The site displays third-party advertisement frames that can float above page elements, so a normal click may be intercepted (ElementClickInterceptedException). safe_click() scrolls the target element to the centre of the viewport and clicks it; if the click is still intercepted, it triggers the click on the element through JavaScript. It is used for the Add to cart buttons.

8. Synchronization and Assertions

- Only explicit waits are used: presence, visibility, clickable, URL contains, title contains and alert presence. time.sleep() is not used anywhere.
- Every assertion carries a descriptive message, so a failure states what was expected and what was found.
- Cart verification compares real application data: the arithmetic check (unit price x quantity = line total) validates the application rather than a fixed value.

9. Screenshots and Execution Reporting

Screenshots are stored in the screenshots folder with the step name and a timestamp in the file name (for example 01_login_success and 05_cart_verified), so evidence is easy to match to the report.

`write_html_report()` produces a single self-contained HTML file, `reports/execution_report.html`, that needs no server or plugin. It contains:

- The overall PASS or FAIL status and the generation time
- One row per step with status (green for PASS, red for FAIL), message and timestamp
- A screenshot thumbnail for each step that captured one

The console also prints a [PASS] or [FAIL] line for each step as it is recorded.

10. How to Run the Script

```
# 1. Install the dependency
pip install -r requirements.txt

# 2. Run the automation
python capstone_assignment_1.py
```

Google Chrome must be installed. Selenium Manager downloads the matching driver automatically on the first run, which requires an internet connection. After the run, open `reports/execution_report.html` in a browser to view the results.

11. Challenges and Design Decisions

Challenge	Resolution
The site has no fixed public login credentials	<code>step_login()</code> registers the account on the first run and, because the email is fixed, detects the "Email Address already exist!" message on later runs and logs in directly instead.
The site styles headings in upper case, and Selenium returns the rendered text ("SEARCHED PRODUCTS")	Text checks are case-insensitive, and the heading locator is an XPath anchored on the actual page text.
Several headings share the same CSS class, so a class-based locator matched the wrong heading	The Searched Products heading is located by its text content instead of by class name.
The same product name appears in different letter case on the product page and in the cart	Cart rows are matched using lower-cased product names, while messages still show the display name.
Advertisement frames intercept clicks on Add to cart	<code>safe_click()</code> centres the element and falls back to a JavaScript click when the native click is intercepted.
Quantity cannot be edited inside the cart on this site	The quantity is updated on the product detail page before adding to the cart, and the result is verified in the cart table.
Driver and browser version drift	Selenium Manager resolves the correct driver automatically.
Keeping the project simple	A single script with function-per-step structure and one dependency, instead of a larger framework.

12. Conclusion

The script covers all ten requirements of the capstone project. It launches the browser, authenticates, searches for a product, adds products to the cart, updates the quantity, verifies names, quantities and line totals in the cart, captures screenshots after each step, takes its inputs from a single test-data block, handles alerts and click-intercepting overlays, and generates a structured HTML execution report even when a step fails. Keeping the solution to one readable script with explicit waits and meaningful assertions makes it easy to run, review and extend, for example with checkout or multi-product scenarios.

13. Appendix: Source Code and Demonstration

GitHub Link: https://github.com/Mayukh27/Wipro_Python_Automation

Video Demonstration Link: https://drive.google.com/drive/folders/13hHv1CmXM-gd4oN5LhwraoYLWVpvDJHe?usp=drive_link

The project source is capstone_assignment_1.py; the generated execution report and screenshots are created in the reports and screenshots folders on each run.